

Chapter 3

Requirements Elicitation

Contents

- Introduction
- Types of Requirements
- What, why, when, who, & How of Requirement Engineering?
- Requirement Engineering Activities
 - Feasibility studies
 - Requirements elicitation and analysis
 - Requirement specification
 - Characteristics of Requirements
 - Requirements validation
 - The requirements document
 - Requirements change management

Introduction

- If you are required to develop a library information software system, what will you do firstly?
 - What is the scope and boundary of the software?
 - What functions that the system may have?
 - What performances that the system may have?
- *Software requirements* are the descriptions of the system services (desired & essential) and constraints (on System operation and software development process).
- Software requirements example for library system
 - *Services*: borrow book, renew book, ...
 - *Constraints*:
 - *Performance*: the time of query book must be lower 1 second
 - *Schedule*: development period is 6 months

Types of Requirements

- Software requirements are often classified as functional requirements, non-functional requirements or domain requirements
- **Functional requirements**
 - Describes functionality or system services.
 - Examples:
 - The user shall be able to search all of the initial set of databases.
 - The system shall provide *appropriate viewers* for the user to read documents in the document store.
 - Every order shall be allocated a unique identifier, which the user shall be able to copy to the account's permanent storage area.
- Answering the following questions on next slide to identify these types of requirements

Functional Requirements...

- What *inputs* the system should accept?
- What *outputs* the system should produce?
- What data the system should *store* that other systems might use?
- What *computations* the system should perform?
- The *timing and synchronization* of the above
- Functional requirement specification of a system should be both *complete* and *consistent*
- However, practically it is impossible for large complex systems to achieve requirement completeness and consistency
- Reason:
 - Easy to mistakes and omissions for large complex systems
 - Different stockholders have different & often inconsistent needs

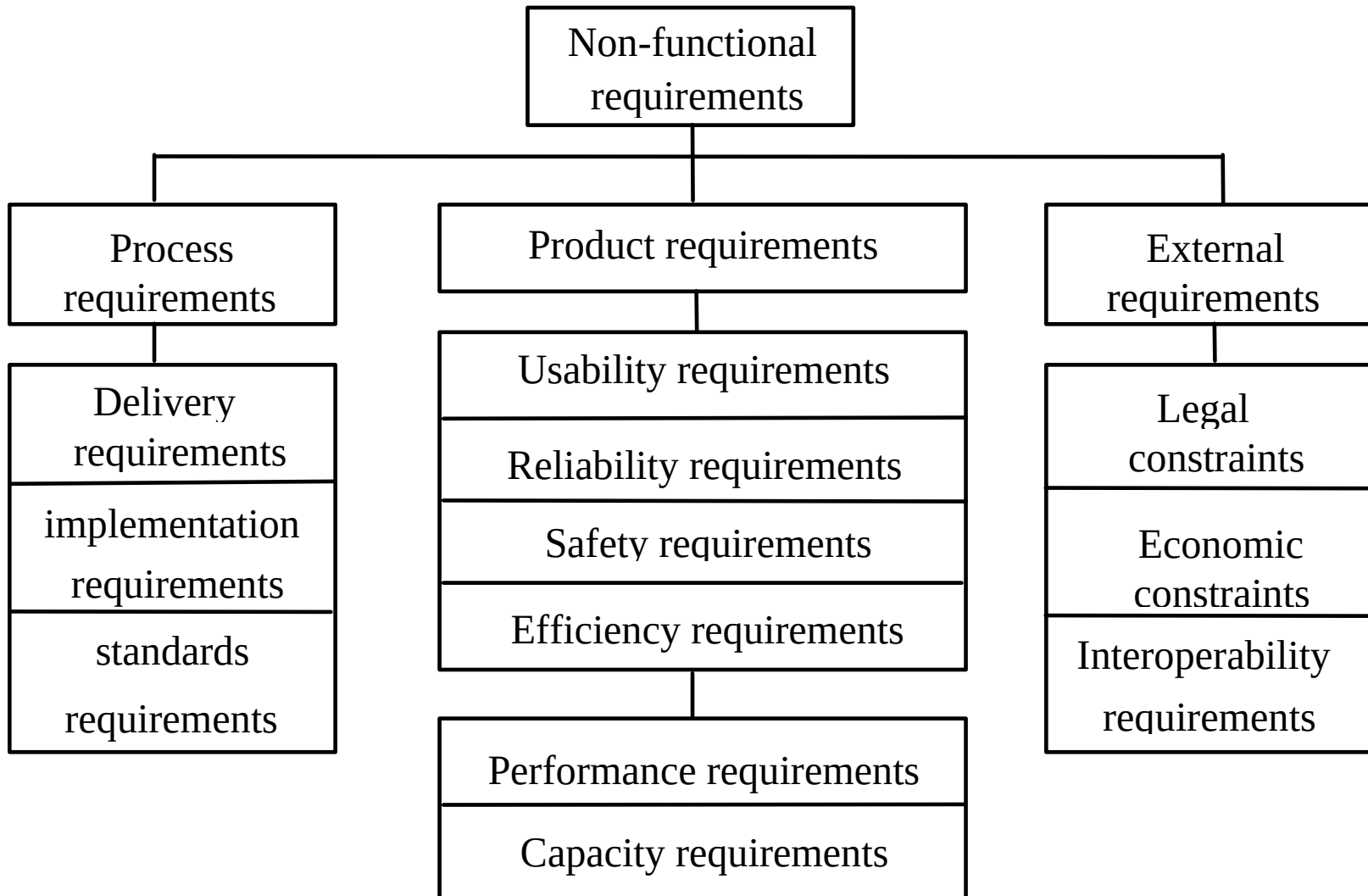
Non-Functional Requirements

- Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.
- Examples
 - reliability, response time, storage requirements, I/O device capability, system representations, etc.
 - Mandating a particular CASE system, programming language or development method
- NFRs are not directly concerned with specific functions delivered by the system.
- However, non-functional requirements may be more critical than functional requirements.
 - That is if they are not met, the system may be useless.

Non-Functional Requirements...

- Non-functional requirements can be
 - *Product requirements*: Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.
 - *Example*: The system shall not fail more than two times per week.
 - *Process requirements*: are constraints placed upon the development process of the system
 - *Example*: The system must be developed using the XYZ suite of CASE tools
 - *External requirements*: Requirements which arise from factors which are external to the system and its development process .
 - *Example*: The system shall not disclose any personal information about customers apart from their name and reference number to the operators of the system

Non-Functional Requirements...



Non-Functional Requirements...

- Non-functional requirements may be very difficult to state *precisely* and often they *conflicts* one another.
- Example; Spacecraft system
 - To minimise weight, the number of separate chips in the system should be minimised
 - To minimise power consumption, lower power chips should be used
 - However, using low power chips may mean that more chips have to be used.
- To resolve these problems
 - Refer the goal - a general intention of the user such as ease of use
 - Optimise
 - Make the requirements *verifiable*

Domain requirements

- Requirements that come from the application domain of the system and that reflect characteristics of that domain.
 - They don't from specific needs of system users.
- They usually include specialized terminologies or reference to domain concept.
- They can be functional or non-functional requirements.
- *Example*
 - There shall be a standard user interface to all databases which shall be based on the Z39.50 standard.
- Problems with domain requirements
 - Understandability
 - Implicitness

User and System requirements

- Software requirements are defined at various levels of detail and granularity.
- Requirements at different level of detail also mean to serve different purposes.
- **Business Requirements:**
 - These are used to state the high-level business objective of the organization or customer requesting the system or product.
 - They are used to document main system features and functionalities without going into their nitty-gritty details.
 - They are captured in a document describing the project vision and scope.

User and System requirements

- **User Requirements:**

- User requirements add further detail to the business requirements.
- They are called user requirements because they are written from a user's perspective and the focus of user requirement describe tasks the user must be able to accomplish in order to fulfill the above stated business requirements.
- They are captured in the requirement definition document.

- **System Requirements:**

- System requirements are expanded versions of the user requirements that are used by software engineers as the starting point for the system design.
- They add detail and explain how the user requirements should be provided by the system

What is Requirement Engineering?

- To understand customers' requirements about the software to be developed is extremely important.
- However,
 - Customers are unable to express requirements explicitly
 - Typically, they can not tell you what they want clearly.
 - The requirements that customers or end-users present are often: *incomplete*, *inaccurate*, and *inconsistent*.
 - Stakeholders (business and technical groups) speak different languages.
 - Software requirements are often extremely complex and large scale.
- These are some of the reasons why engineering requirements is needed

What is Requirement Engineering?...

- Requirement engineering is the process of establishing the services that are required of the system, and, the constraints under which it operates and is developed.
 - It covers all activities involved in discovering, documenting and maintaining a set of requirements for a computer-based system.
 - It helps software engineers to better understand the problem they will work to solve.
 - It can influence the development: cost, time, effort and quality.
- The goal of requirement engineering is to:
 - obtain and understand software requirements in a *complete*, *consistent* and *accurate* way, and generate software requirement specification (SRS) document.

Why Requirement Engineering?

- In spite of new and effective software engineering techniques, software systems
 - Are delivered late and over budget
 - Do not do what really users want
 - Are prone to failure
- Some key aspects of successful software development are
 - User input and involvement
 - Effective management and support
 - Resources
 - **Clearly defined, complete requirements**
 - Numerous software engineering studies show this repeatedly
 - 40-60% of all defects found in software projects can be traced back to poor requirements.

Why Requirement Engineering?...

The hardest single part of building a software system is deciding precisely what to build. No other part of the conceptual work is as difficult as establishing the detailed technical requirements, including all the interfaces to people, to machines, and to other software systems. No part of the work so cripples the resulting system if done wrong. No other part is more difficult to rectify later. (Brooks, 1987)

No Silver Bullet: Essence and Accidents of Software Engineering.

- Generally, defining and applying good, complete requirements is hard to work, and success in this endeavor has eluded many software engineers.

Stakeholders

- System Stakeholders are people or organizations who will be affected by the system and who have direct or indirect influence on the system requirements.
- In order to develop a good requirement document, it is imperative to involve all kinds of user in the requirement engineering process.
- You need to identify the stakeholders in a system and discover their requirements.
- If you don't do, you may find that they insist on changes during the system development or after it has been delivered for use.
- Stakeholders have a tendency to state requirements in very general and vague terms
 - different stakeholders have different requirements – sometimes even conflicting.

Stakeholders...

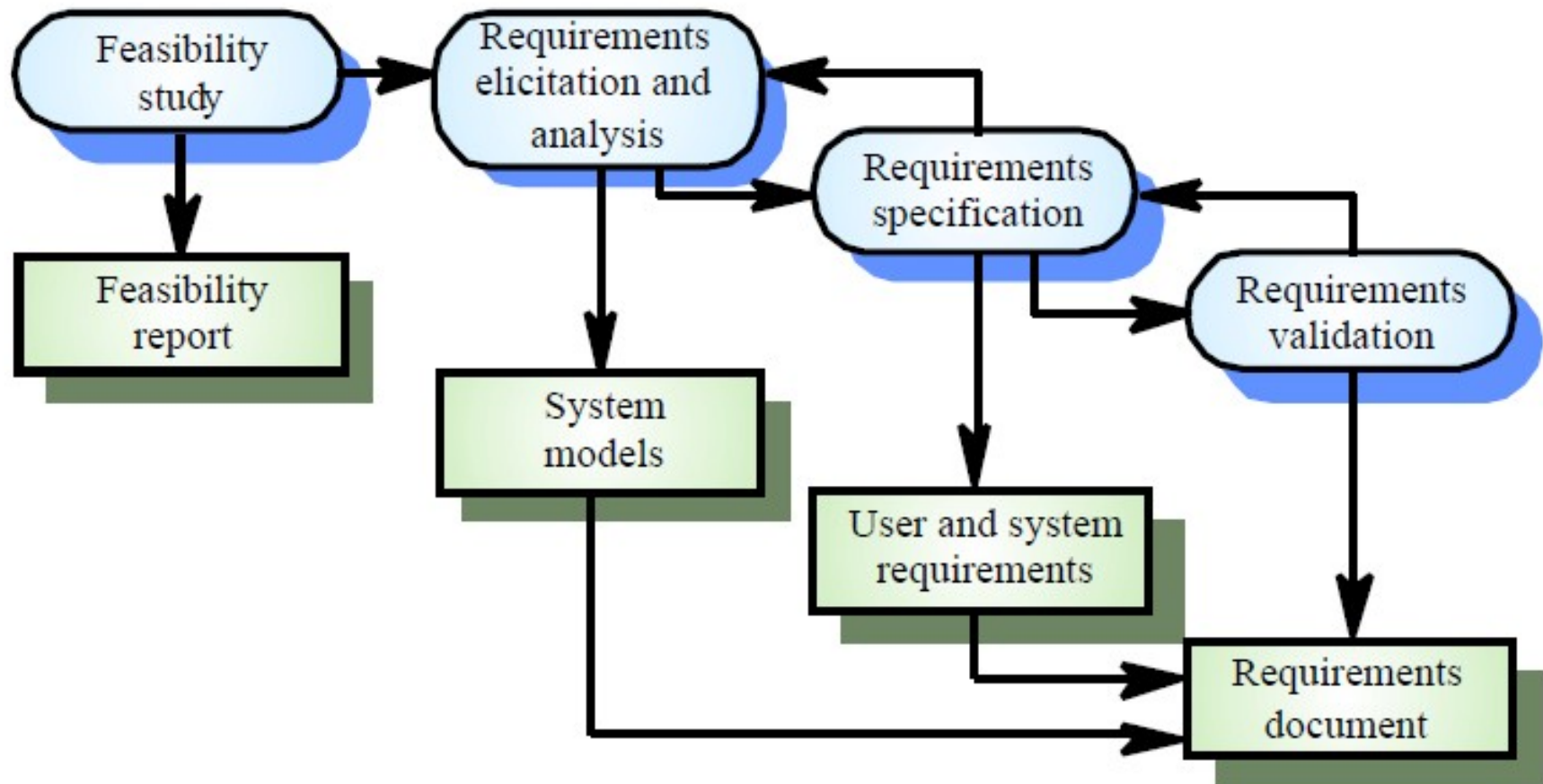
- Example
 - For an automated railway signaling system (a system used to control railway traffic safely) possible stakeholders are
 - Train company operators responsible for running the system
 - Train crew
 - Railway managers
 - Passengers
 - Equipment installation and maintenance engineers
 - Safety certification authorities

Requirement Engineering Activities

- Software requirements engineering is made up of two major processes: requirements development and requirements management.
 - Requirements development encompasses all of the activities involved in eliciting, analyzing, specifying, and validating the requirements.
 - It occur during the early concept and requirements phases of the life cycle.
 - Requirements management is the process of understanding and controlling changes to system requirements and maintaining links between dependent requirements.
- The activities used for requirement engineering vary widely depending on the application domain, the people involved and the organisation developing the requirements.

Requirement Engineering Activities...

- However, there are a number of generic activities common to all processes: *Feasibility studies*, *Requirements elicitation and analysis*, *Requirement specification*, *Requirements validation*, *Requirements change management*



Feasibility study

- A feasibility study decides whether or not the proposed system is worthwhile.
- A short focused study that checks if the system;
 - contributes to organisational objectives?
 - can be engineered using current technology and within budget?
 - can be integrated with other systems that are used?
- Unless the answers for the above questions are affirmative it has no real value to the business.
- Feasibility study involves *information assessment, information collection and reporting.*

Requirements elicitation & Analysis...

- Involves finding out the services that the system should provide and the system's operational constraints.
- Is the most difficult, most critical, most error-prone, and most communication-intensive aspect of software development.
- Why Requirements elicitation and analysis is a difficult process?
 - Stakeholders often don't really know what they want from the computer system.
 - Stakeholders naturally express requirements in their own terms.
 - Different stakeholders have different requirements.
 - Political factors may influence the requirements of the system.
 - The requirements change during the analysis process. New stakeholders may emerge and the business environment change
- Requirements elicitation & analysis process activities involves:
 - Domain understanding

Requirements elicitation & Analysis...

- Requirement collection
- Classification (into coherent clusters)
- Conflict resolution and identification of unpractical requirements
- Prioritization
- Requirement checking (e.g. Eliminate, combined and modified requirements)
- Different models may be produced during the requirements analysis activity.
- Requirements analysis may involve three structuring activities which result in these different models
 - *Partitioning*. Identifies the structural (part-of) relationships between entities
 - *Abstraction*. Identifies generalities among entities
 - *Projection*. Identifies different ways of looking at a problem

Requirements elicitation & Analysis...

- There are various requirement elicitation and analysis techniques. Some of these are:
 - Interview
 - Prototyping
 - Ethnography
 - Scenarios
 - Session Videotaping
 - Brainstorming
 - Delphi Technique
 - Viewpoints
 - Use cases
- **Interview**
 - In formal or informal interviewing, the RE team puts questions to stakeholders about the system that they use (if any) and the system to be developed.
 - Ask about specific details and about the stockholder's vision for the future
 - Ask if they have alternative ideas

Requirements elicitation & Analysis...

- Ask for other sources of information
- Ask them to draw diagrams
- Don't ask questions like 'what do you want'.
- Interviewers should be open-minded, willing to listen to stakeholders and should not have pre-conceived ideas about the requirements.
- There are two types of interview
 - Closed interviews where a pre-defined set of questions are answered.
 - Open interviews where there is no pre-defined agenda and a range of issues are explored with stakeholders.
- Normally a mix of closed and open-ended interviewing are used together.
- Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system.

Requirements elicitation & Analysis...

- Interviews are not good for understanding domain requirements.
 - Difficult to understand specific domain terminology;
 - Some domain knowledge is so familiar that people find it hard to articulate or think that it isn't worth articulating.
- **Brainstorming**
 - process of systematic and liberal generation of a large volume of ideas from a number of participants.
 - In a brainstorming session you (moderator) gather together a group of people, create a stimulating and focused atmosphere, and let people come up with ideas without risk of being ridiculed.
 - The facilitator notes down all ideas on a whiteboard.
 - *Joint Application Development (JAD)* is a technique based on intensive brainstorming sessions.

Requirements elicitation & Analysis...

- **Prototyping**

- Software customers and end users usually find it very difficult to express their real requirements
- Prototyping provide insight to the new functionalities by demonstrating a new prototype
 - The simplest kind: *paper prototype*.
 - a set of pictures of the system that are shown to users in sequence to explain what would happen
 - The most common: a mock-up of the system's UI
 - Written in a rapid prototyping language
 - Does *not* normally perform any computations, access any databases or interact with any other systems
 - May prototype a particular aspect of the system
- Can be combined with other techniques

Requirements elicitation & Analysis...

• Delphi Technique

- is an iterative technique in which information is exchanged in a written form until a consensus is reached.
- For example
 - participants may write down their requirements, sorted in order of importance.
 - The sets of requirements obtained are distributed to all participants, who reflect on them to obtain a revised set of requirements.
 - The procedure will be repeated several times until sufficient consensus is reached.

• Ethnography

- Ethnography is an observational technique that can be used to understand social and organizational requirements.
- A social scientists spends a considerable time observing and analysing how people actually work.

Requirements elicitation & Analysis...

- People do not have to explain or articulate their work.
- Social and organisational factors of importance may be observed.
- Ethnographic studies have shown that work is usually richer and more complex than suggested by simple system models.
- The problem with ethnography is that it studies existing practices which may have some historical basis which is no longer relevant.
- **Viewpoints**
 - Viewpoints are a way of structuring the requirements to represent the perspectives of different stakeholders.
 - Stakeholders may be classified under different viewpoints.
 - This multi-perspective analysis is important as there is no single correct way to analyse system requirements of all stakeholders.
 - *Advantage:* Provides a framework for discovering conflicts in the requirements proposed by different stakeholders.

Requirements elicitation & Analysis...

- Principal stages of Viewpoint-oriented elicitation & Analysis:
 - *Viewpoint identification (e. g. by brainstorming)*
 - Discover viewpoints which receive system services and identify the services provided to each viewpoint
 - *Viewpoint structuring*
 - Group related viewpoints into a hierarchy. Common services are provided at higher-levels in the hierarchy
 - *Viewpoint documentation*
 - Refine the description of the identified viewpoints and services
 - *Viewpoint-system mapping*
 - Transform the analysis to an object-oriented design

Requirements elicitation & Analysis...

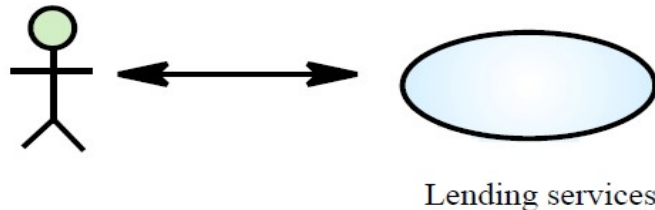
- **Scenarios**

- are descriptions of how a system is used in practice.
- They are helpful in requirements elicitation as people can relate to these more readily than abstract statement of what they require from a system.
- Scenarios are particularly useful for adding detail to an outline requirements description
- Scenarios descriptions include
 - A description of the starting situation;
 - A description of the normal flow of events;
 - A description of what can go wrong;
 - Information about other concurrent activities;
 - A description of the state when the scenario finishes.

Requirements elicitation & Analysis...

- **Use cases**

- Use-cases are a scenario based technique in the unified modelling language (UML) which identify the actors in an interaction and which describe the interaction itself.
 - UML is a standard for modelling object-oriented software requirements and design.
- A set of use cases should describe all possible interactions with the system.
- Sequence diagrams may be used to add detail to use-cases by showing the sequence of event processing in the system.



Software Requirement Specification

- In this activity complete description of the behavior of the system is developed.
- The requirements specification document states in precise and explicit language those functions and capabilities a software system must provide, as well as stating any required constraints by which the system must abide.
- Recommended approaches for the specification of software requirements are described by IEEE 830-1998.
- Guidelines for writing user requirement Specification
 - Must be written in natural languages because they have to be understood by people who are not technical experts.
 - Invent a standard format and use it for all requirements.

Software Requirement Specification...

- Use language in a consistent way. Use *shall* for mandatory requirements, and *should* for desirable requirements.
- Use text highlighting to identify key parts of the requirement.
- Avoid the use of computer jargon.
- However, since user requirements are written in natural language various problems may arise ;
 - Lack of clarity,
 - Requirements confusion,
 - Requirements amalgamation.
- Guidelines for writing system requirement Specification
 - Though this type of requirements are written for developers natural language is often used to write system requirement specification as well as user requirements.

Software Requirement Specification...

- However, because system requirements are more detailed than user requirements, natural language specification can be confusing and hard to understand.
- Alternatives to natural language specification
 - *Structure natural language*
 - depends on defining standard forms or templates to express the requirements specification.
 - The form contains: Function, description, input, source, output, precondition, post condition, etc
 - *Graphical notations*
 - A graphical language, supplemented by text annotations is used to define the functional requirements for the system.
 - Examples: Structured Analysis and Design Technique (SADT) and use-case descriptions.

Software Requirement Specification...

- ***Design Description Language:***
 - uses a language like a programming language but with more abstract features to specify the requirements by defining an operational model of the system.
 - E.g. Program Description language (PDL)
- ***Mathematical specifications***
 - Notations based on mathematical concepts such as finite-state machines or sets.
 - Is unambiguous specifications which reduce the arguments between customer and contractor about system functionality.
 - However, most customers don't understand formal specifications and are reluctant to accept it as a system contract.

More on characteristics of requirements...

- **Correct** – A requirement has to be correct.
 - The information used to create it has to be correct and has to reflect reality.
 - In general, the development team is not going to know if the requirements are correct, so have another experienced analyst or engineer review the requirements with you before finalizing them.
- **Necessary** – A requirement has to be necessary.
 - If it wasn't included, what's the worst that could happen? If you and the users are willing to accept those consequences, then it probably isn't really necessary.
- **Focus**(not amalgamated) – A requirement has to focus on one thing.
 - Don't write one requirement that specifies multiple needs – instead, break it up into several requirements.
 - **Wrong** – “The meter shall display a minimum of -999 and a maximum of 999 volts.”

More on characteristics of requirements...

- **Feasible** – A requirement must be feasible/practical.
 - If you cannot implement it with the budget, schedule, resources, and other limitations that are available to your project, then either it must be dropped or the project has already failed. Designers and developers need to work with the requirements analysts to figure this out.
- **Verifiable** – A requirement has to be testable/testable.
 - Once a solution for a requirement has been implemented, you need some way to know if the design and implementation are correct. If you can't test the requirement, then the product is indeterminate – you can't prove that what you built is right, and you may never get acceptance from the customer.
 - *Wrong: The TVRS shall complete storage of data within a reasonable time of the user confirming a “Save” sequence*
 - *Correct: The TVRS shall complete storage of data within 5 seconds of the user confirming a “Save” sequence, 80% of the time*

More on characteristics of requirements...

- **Precise** – requirement should be evident and unambiguous
 - The system shall accept valid employee ID numbers from 1 to 9999.
 - Are all numbers between 1 and 9999 valid? Is 2 OK as well as 0002?
- **Clear** – A requirement has to be clear and unambiguous.
 - The maximum speed of the treadmill shall be 10 mph.
 - The treadmill speed will not exceed 10 mph
 - The treadmill speed should not exceed 10 mph
- **Deal only with the problem**
 - Requirements should state “what” is required at the appropriate system level, not “how”
- **Not be gold plated**
 - Example: *TVRS(traffic violation report system) will play a piece of classical music during initialization*

Requirements validation

- Concerned with demonstrating that the requirements define the system that the customer really wants and they are error free.
- Requirements error costs are high so validation is very important
 - Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.
- During the requirements validation process, different types of checks should be carried out on the requirements in the requirement document. Some of the checks are
 - Validity
 - Consistency
 - Completeness
 - Realism
 - Verifiability

Requirements validation techniques

- To validate requirements we may use the following requirement validation techniques
 - Requirements reviews
 - Systematic manual analysis of the requirements.
 - Prototyping
 - Using an executable model of the system to check requirements.
 - Test-case generation
 - Developing tests for requirements to check testability.
 - Automated consistency analysis
 - Checking the consistency of a structured requirements description

Requirement Document

- The software requirements document [sometimes called the software requirements specification (SRS)] is the official statement of what is required of the system developers
- It should include both the user requirements for a system and a detailed specification of the system requirements.
- The requirements document has diverse set of users, for example :
 - System customers, Managers, System Engineers, System Test Engineer, System Maintenance Engineers
- Writing SRS
 - Write each clause so that it contains only one requirement
 - Avoid having one requirement refer to another requirement
 - Collect similar requirements together

Requirement Document...

- The requirement document should be
 - **Complete** - the set of requirements must be complete.
 - If requirements are missing, then the product will also be incomplete.
 - It is likely that the missing requirements will turn up at inopportune times and cause problems throughout the project life cycle.
 - **Consistent** - set of requirements must be consistent.
 - If there are requirements in your specification that conflict with each other, then the product will not be able to meet all of your requirements.
 - Inconsistencies must be resolved in order for your project to succeed.
 - **Updateable**
 - If you have to change a requirement (create, edit, or delete), you must be able to evaluate the impact of that change on all of the other requirements.

Requirement Document...

- **Traceability** - Each requirement should be contained in a single, numbered paragraph so that it may be referred to in other documents:
 - *Backward traceability* – implies that we know why every requirement exists. I.E. each requirement explicitly references its source in previous documents
 - *Forward traceability* allocation of requirement to analysis/design elements.
- **Prioritization** - Each requirement has to be ranked against the others according to its implementation importance.
- More over SRS should
 - Specify external system behaviour
 - Specify implementation constraints
 - Easy to change
 - Serve as reference tool for maintenance etc

IEEE 830-1998. requirements standard

1. Introduction to the Document

- 1.1 Purpose of the Product
- 1.2 Scope of the Product
- 1.3 Acronyms, Abbreviations, Definitions
- 1.4 References
- 1.5 Outline of the rest of the SRS

2. General Description of Product

- 2.1 Context of Product
- 2.2 Product Functions
- 2.3 User Characteristics
- 2.4 Constraints
- 2.5 Assumptions and Dependencies

3. Specific Requirements

- 3.1 External Interface Requirements
 - 3.1.1 User Interfaces
 - 3.1.2 Hardware Interfaces
 - 3.1.3 Software Interfaces
 - 3.1.4 Communications Interfaces

3.2 Functional Requirements

- 3.2.1 Class 1
- 3.2.2 Class 2

...

3.3 Performance Requirements

3.4 Design Constraints

3.5 Quality Requirements

3.6 Other Requirements

4. Appendices

5. Index

Requirements management

- Requirements management is the process of managing changing requirements during the requirements engineering process and system development.
- Requirements change for the following reasons:
 - The priority of requirements from different viewpoints changes during the development process.
 - System customers may specify requirements from a business perspective that conflict with end-user requirements.
 - The business and technical environment of the system changes during its development.
- Requirement change management should be applied to all proposed changes to the requirements so that changes are treated consistently and the changes to the requirements document are made in a controlled way .

Requirement change management...

- Principal stages to change management process are
 - *Problem analysis* - Discuss requirements problem and propose change;
 - *Change analysis and costing* - Assess effects of change on other requirements;
 - *Change implementation* - Modify requirements document and other documents to reflect change.
- From an evolution perspective, requirements fall into two classes
 - ***Enduring requirements:*** Stable requirements derived from the core activity of the customer organisation. May be derived from domain models
 - *E.g.* a hospital will always have doctors, nurses, etc.
 - ***Volatile requirements:*** Requirements which change during development or when the system is in use.
 - In a hospital, requirements derived from health-care policy

Requirements management planning

- During the requirements engineering process, you have to plan and decide on:
 - *Requirements identification*
 - How requirements are individually identified;
 - *A change management process*
 - The process followed when analysing a requirements change;
 - *Traceability policies*
 - The amount of information about requirements relationships that is maintained;
 - *CASE tool support*
 - The tool support required to help manage requirements change

System Models: Contents

- Introduction
- UML
- Context models
(Architectural models , Process Model)
- Behavioural models
(DFD, State machine models)
- Data models
(Semantic data models, Data dictionaries)
- Object models
(inheritance, interaction, aggregation, use case)
- CASE workbenches

Introduction

- User requirements must be written in natural languages because they have to be understood by people who are not technical experts.
- However, more detailed system requirements may be expressed in a more technical way.
- One widely used technique is to document the system specification as a set of system models.
- Because system models have graphical representations, they are more understandable than detailed natural language descriptions of the system requirements.

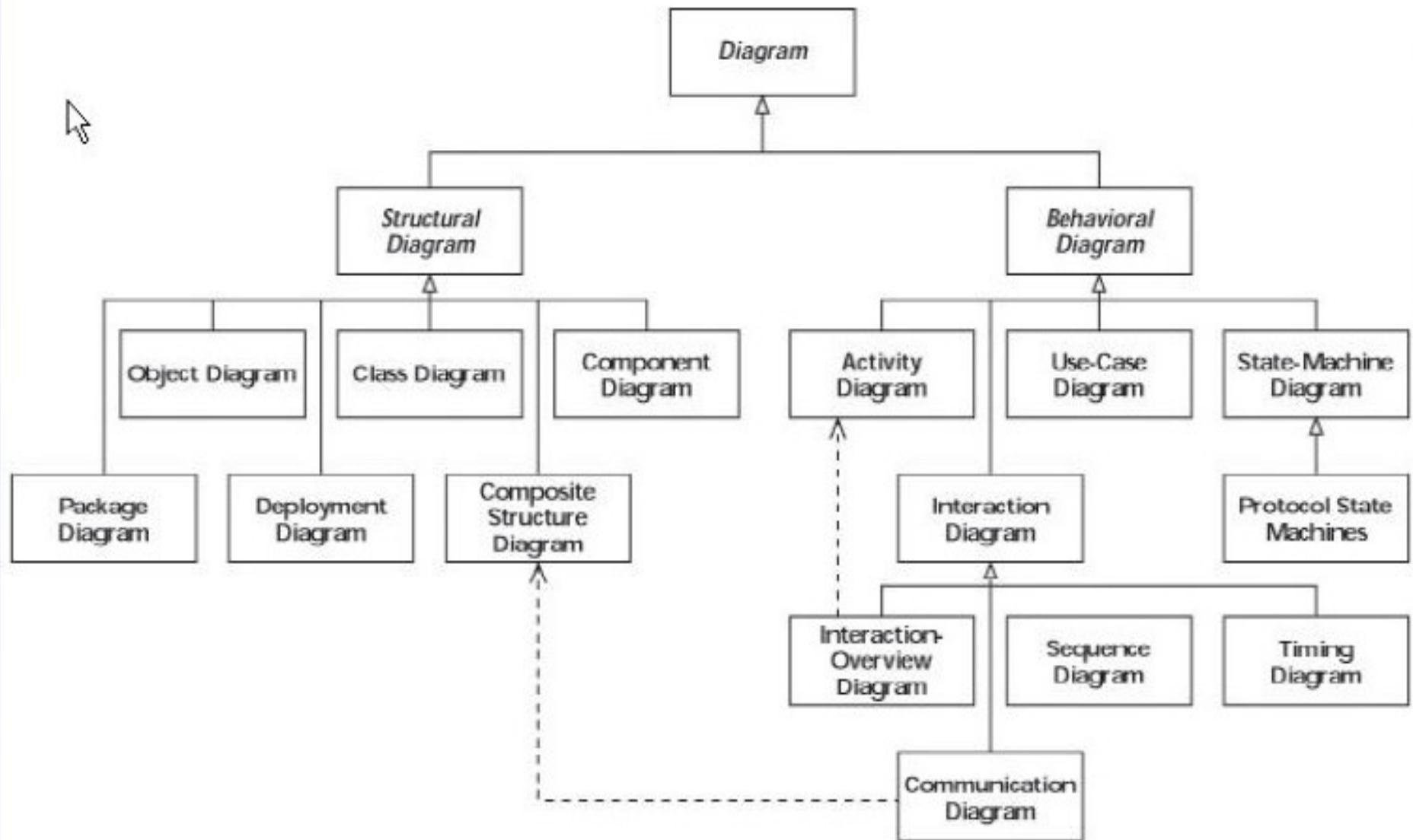
System modelling

- System models are abstract descriptions of systems whose requirements are being analysed
- Different types of models might be produced during requirement analysis process.
 - ***Data processing model***: showing how the data is processed at different stages.
 - ***Composition model***: composition/aggregation model showing how entities are composed of other entities.
 - ***Architectural model***: showing principal sub-systems.
 - ***Classification model***: Object class/inheritance diagrams showing how entities have common characteristics.
 - ***Stimulus-response model***: showing the system's reaction to events.

Unified Modelling Language (UML)

- The UML is a standard representation devised by the developers of widely used object-oriented analysis and design methods.
- It is a graphical language for specifying, visualizing, constructing, documenting the artifacts of software systems
- It has become an effective standard for object-oriented modelling.
- UML is categorized into two:
 - **Structural diagrams** – Used to describe the building blocks of the system – features that do not change with time. These diagrams answer the question – What's there?
 - **Behavioral diagrams** – Used to show how the system evolves over time (responds to requests, events, etc.)

UML Diagrams

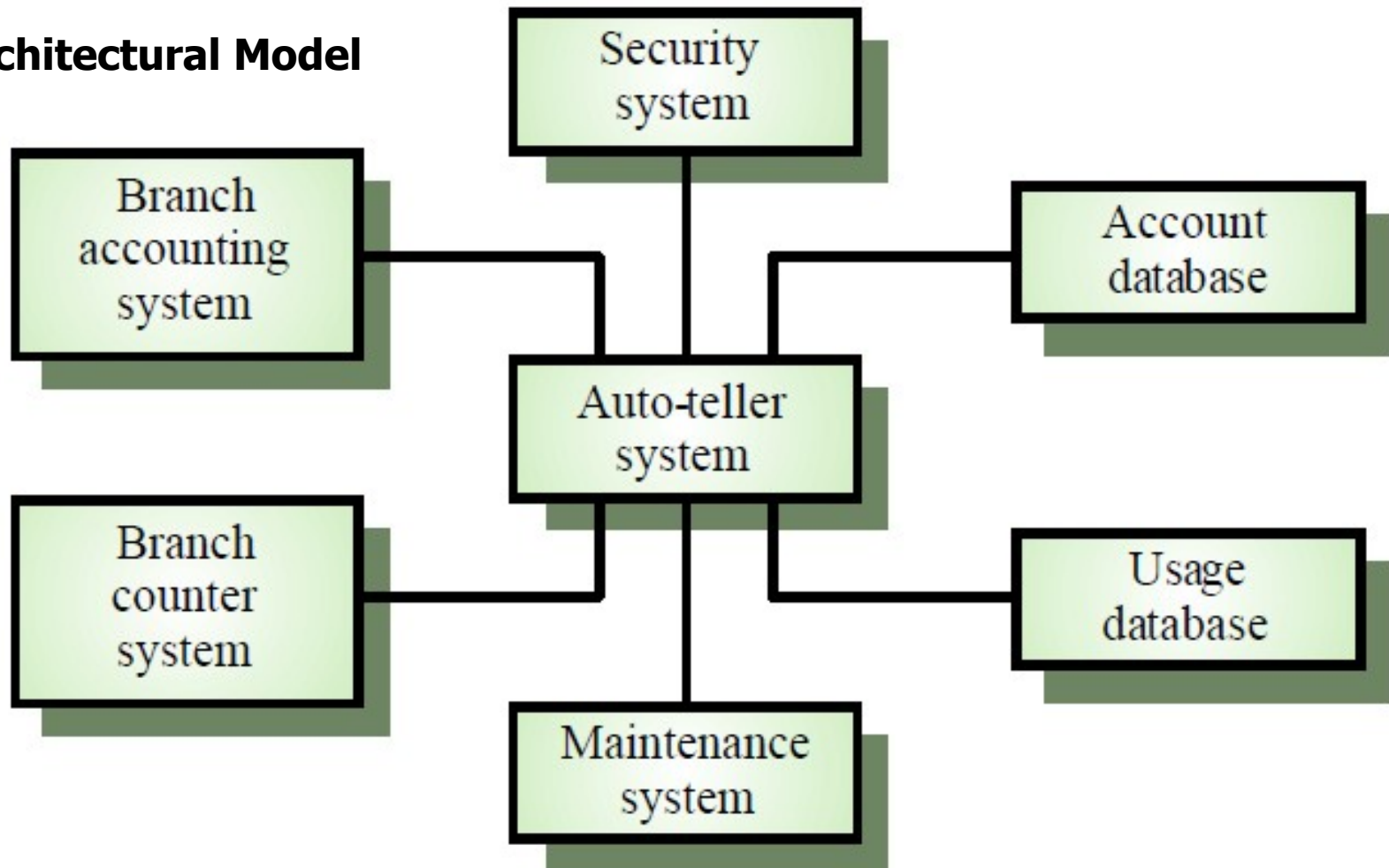


Context models

- Context models are used to illustrate the operational context of a system.
 - They show what lies outside the system boundaries.
- Social and organisational concerns may affect the decision on where to position system boundaries.
- Once decisions have been made on boundaries of the system, context and dependencies that the system has on its environment is defined
- Producing a simple architectural models of the system is the first step in this activity .

The context of an ATM system

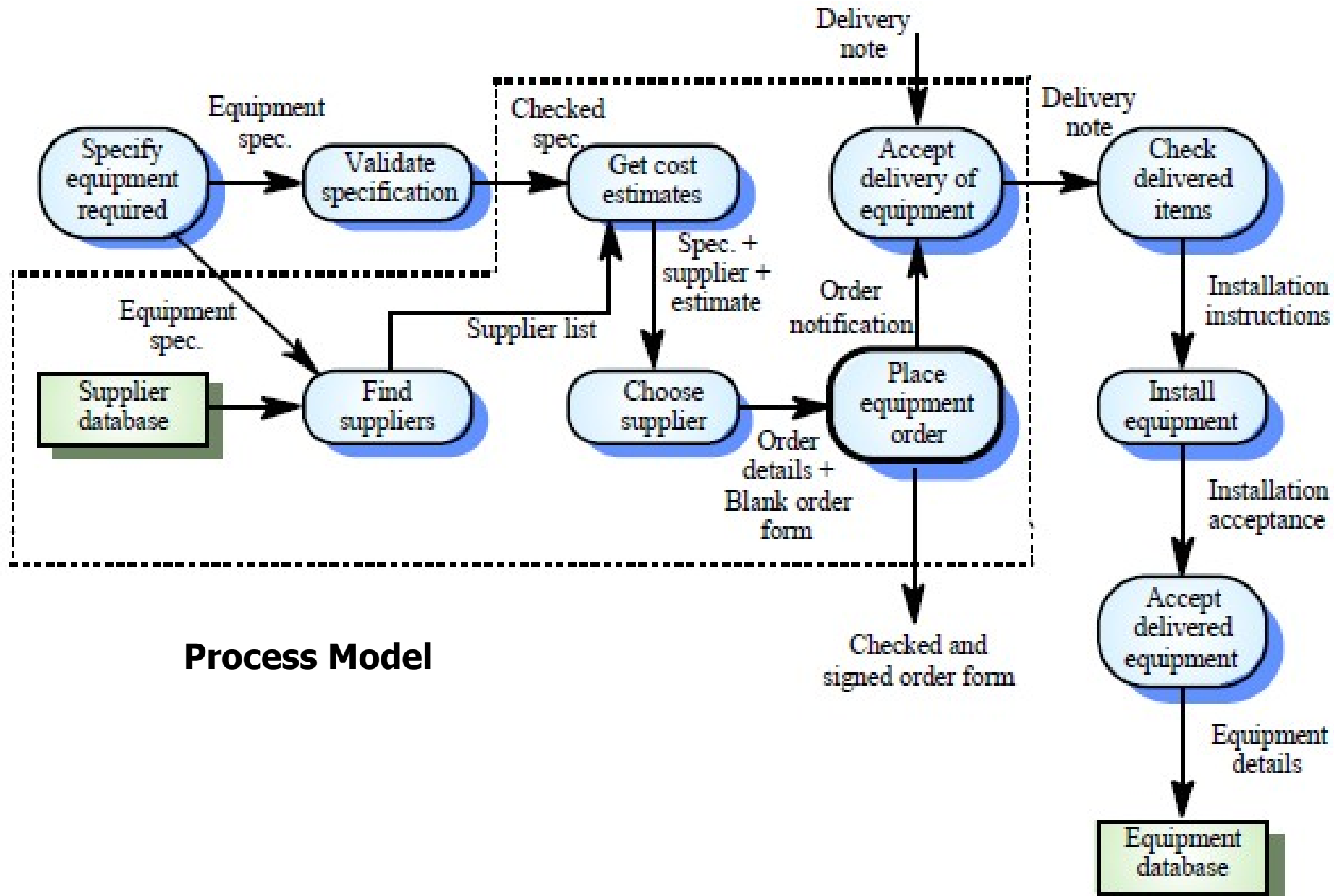
Architectural Model



The context of an ATM system

- Architectural models describe the environment of the system — they do not show the *relationship* between the other systems in the environment and the system that is being specified.
- Hence, simple architectural models are usually supplemented by other models such as process model.
- Process models show the overall process and the processes that are supported by the system.
- Data flow models may be used to show the processes and the flow of information from one process to another.

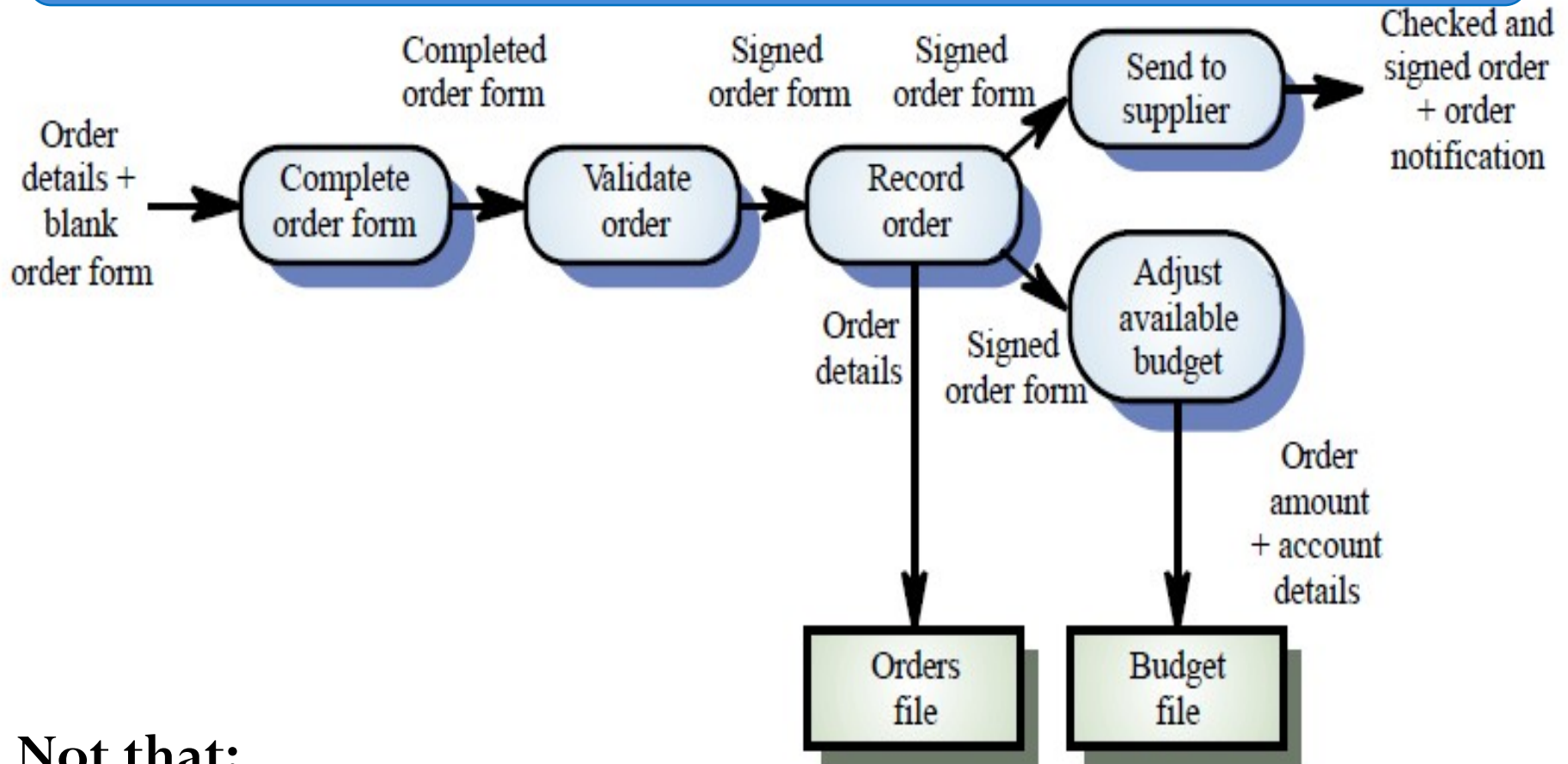
Equipment procurement process



Behavioural models

- Behavioural models are used to describe the overall behaviour of a system.
- Two types of behavioural model are:
 - Data processing models that show how data is processed as it moves through the system;
 - State machine models that show the systems response to events.
- These models show different perspectives so both of them are required to describe the system's behaviour.
- *Data-processing models*
 - Data flow diagrams (DFDs) may be used to model the system's data processing.
 - They show the processing steps as data flows through a system.

Order processing DFD



Not that:

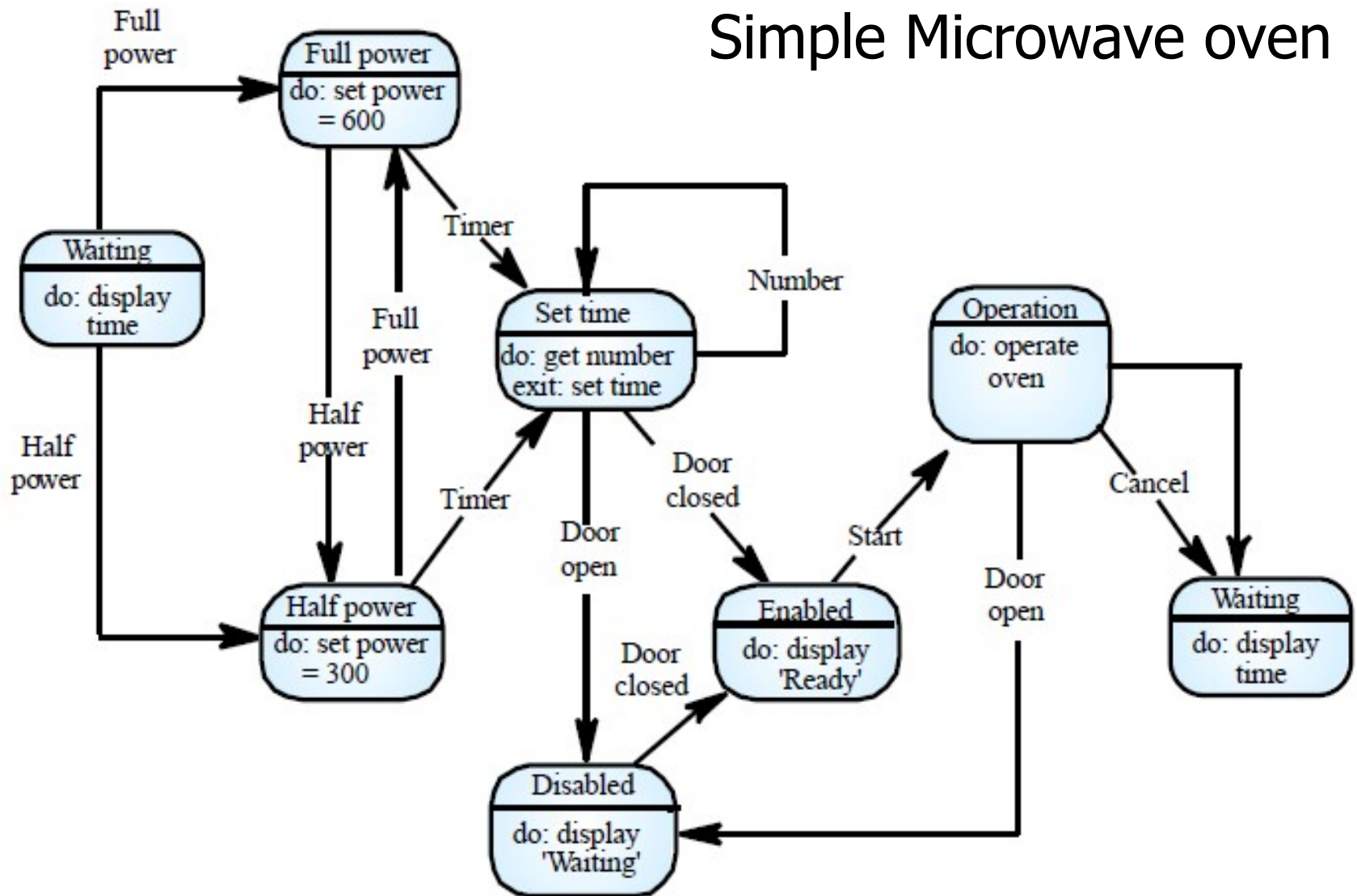
- Data flow diagrams may also be used in showing the data exchange between a system and other systems in its environment.
- multiple DFDs are required to represent a system

State machine models

- These model the behaviour of the system in response to external and internal events.
- *Example 2: Microwave oven model*
 - They show the system's responses to stimuli - so are often used for modelling real-time systems.
 - State machine models show system states as nodes and events as arcs between these nodes.
 - When an event occurs, the system moves from one state to another.
 - Not that:
 - State machine model does not show flow of data within the system

State machine models

Simple Microwave oven



Microwave oven state description

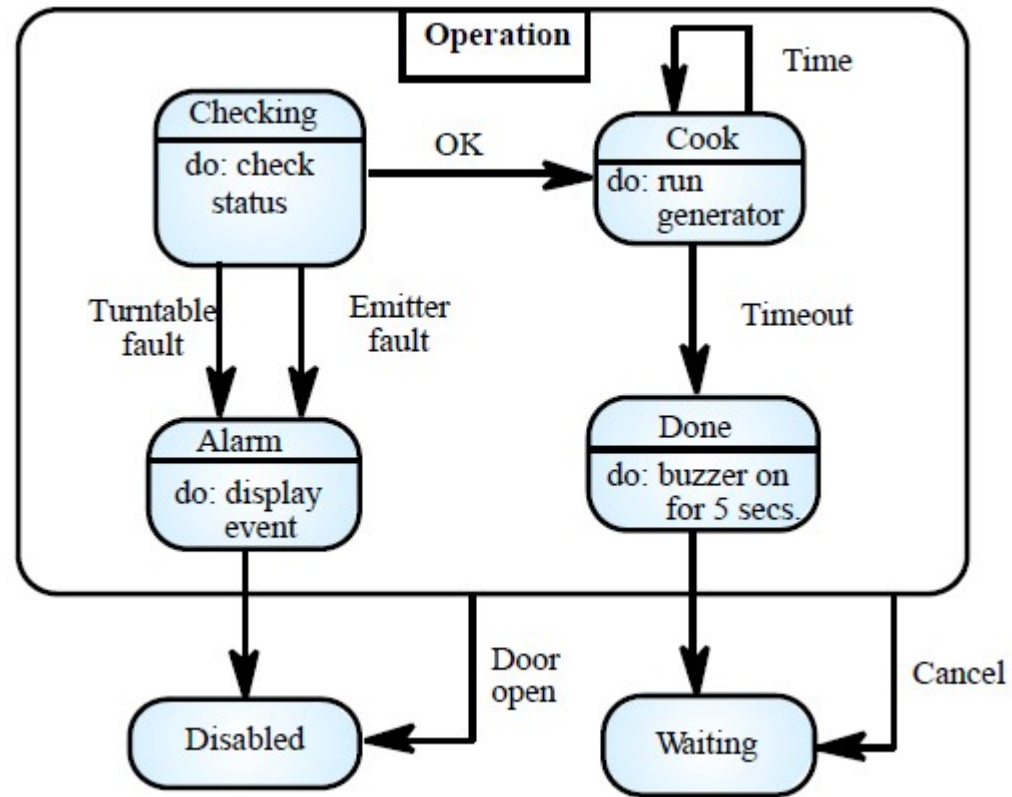
State	Description
Waiting	The oven is waiting for input. The display shows the current time.
Half power	The oven power is set to 300 watts. The display shows 'Half power'.
Full power	The oven power is set to 600 watts. The display shows 'Full power'.
Set time	The cooking time is set to the user's input value. The display shows the cooking time selected and is updated as the time is set.
Disabled	Oven operation is disabled for safety. Interior oven light is on. Display shows 'Not ready'.
Enabled	Oven operation is enabled. Interior oven light is off. Display shows 'Ready to cook'.
Operation	Oven in operation. Interior oven light is on. Display shows the timer countdown. On completion of cooking, the buzzer is sounded for 5 seconds. Oven light is on. Display shows 'Cooking complete' while buzzer is sounding.

Microwave oven stimuli

Stimulus	Description
Half power	The user has pressed the half power button
Full power	The user has pressed the full power button
Timer	The user has pressed one of the timer buttons
Number	The user has pressed a numeric key
Door open	The oven door switch is not closed
Door closed	The oven door switch is closed
Start	The user has pressed the start button
Cancel	The user has pressed the cancel button

Statecharts

- The problem with state machine approach is that the number of possible states increase rapidly.
- For large system models, therefore, some structuring of these state model is necessary.
- Statecharts allow the decomposition of superstate a model into sub-models.
- Can be complemented by tables describing the states and the stimuli



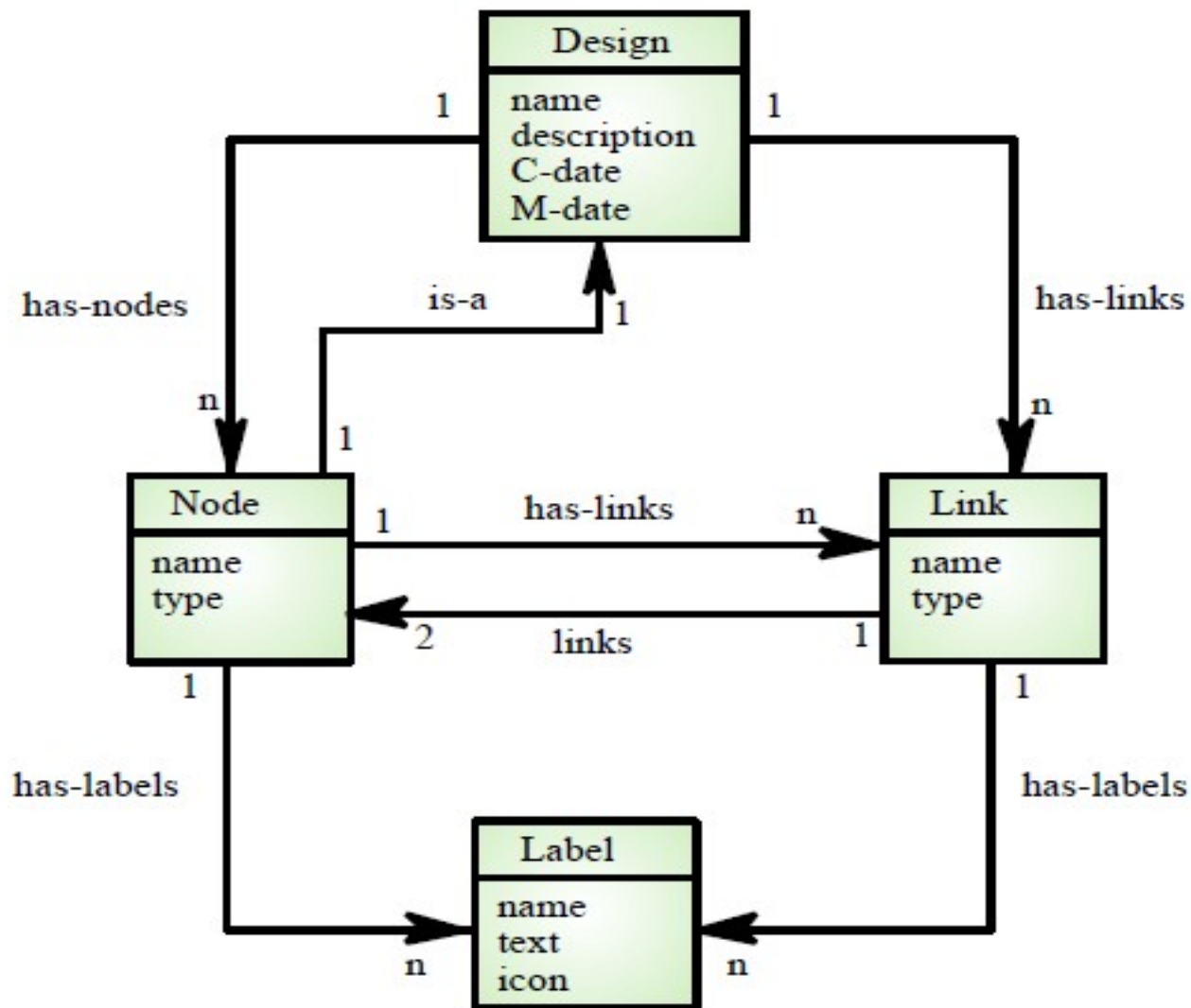
Data Model

- Most large software systems make use of a large database of information.
- Sometimes this database exists independently of the software system.
- Sometimes it is created for the system being developed.
- Defining the logical form of data processed by the system is an important part of system modeling.
- The most widely used data modeling techniques is entity-relation-attribute modeling (ERA modeling).

Semantic data models

- Used to describe the logical structure of data processed by the system.
- An entity-relation-attribute model sets out the entities in the system, the relationships between these entities and the entity attributes
- Widely used in database design.
- Can readily be implemented using relational databases.
- No specific notation provided in the UML but objects and associations can be used.

semantic data model of Software design



Data dictionaries

- Data dictionaries are lists of all of the names used in the system models.
- Descriptions of the entities, relationships and attributes are also included.
- *Advantages*
 - Support name management and avoid duplication
 - Store of organisational knowledge linking analysis, design and implementation;
- Many CASE workbenches support data dictionaries.

Data dictionary entries

Name	Description	Type	Date
has-labels	1:N relation between entities of type Node or Link and entities of type Label.	Relation	5.10.1998
Label	Holds structured or unstructured information about nodes or links. Labels are represented by an icon (which can be a transparent box) and associated text.	Entity	8.12.1998
Link	A 1:1 relation between design entities represented as nodes. Links are typed and may be named.	Relation	8.12.1998
name (label)	Each label has a name which identifies the type of label. The name must be unique within the set of label types used in a design.	Attribute	8.12.1998
name (node)	Each node has a name which must be unique within a design. The name may be up to 64 characters long.	Attribute	15.11.1998

Object models

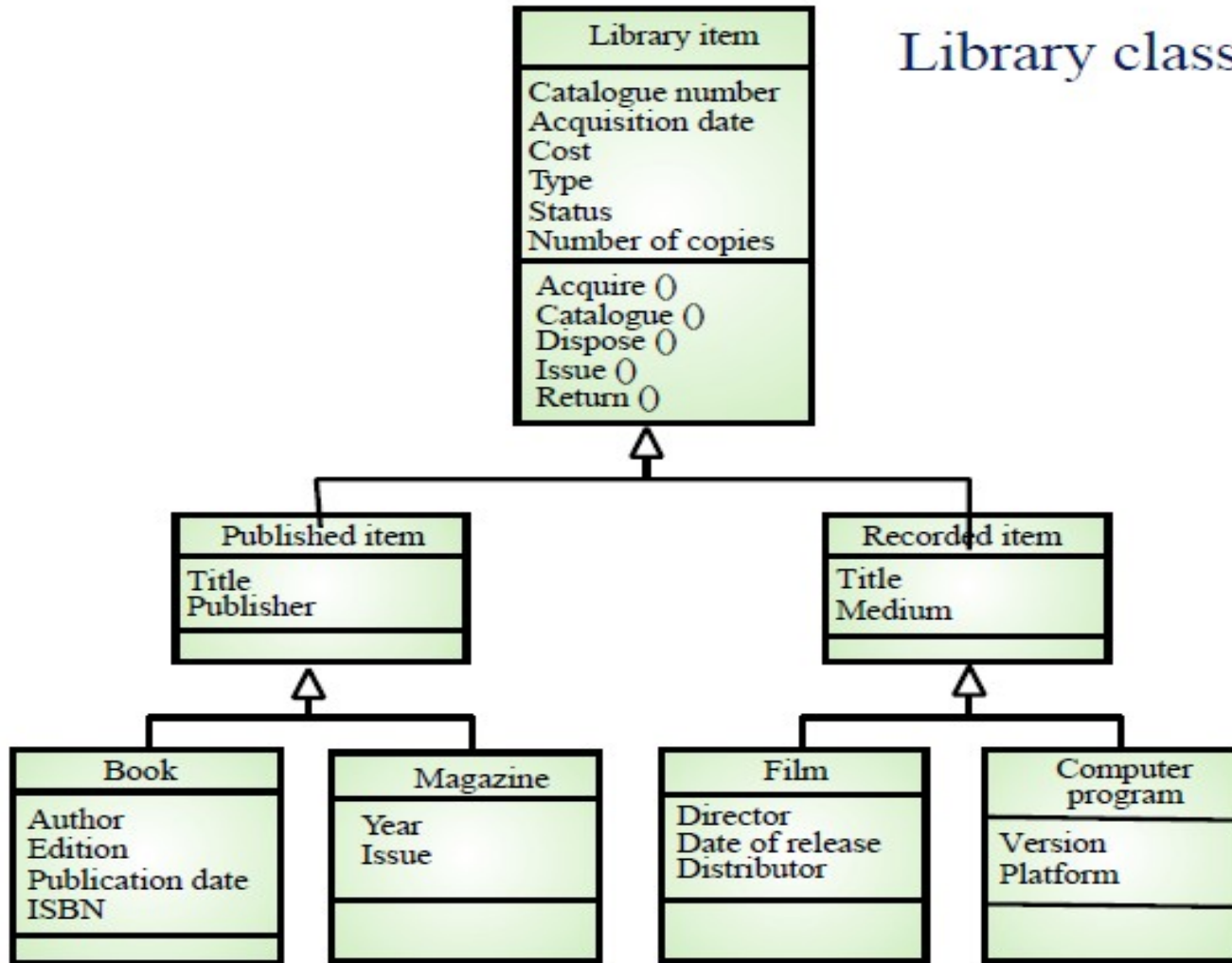
- Object models describe the system in terms of object classes and their associations.
- An object class is an abstraction over a set of objects with common attributes and the services (operations) provided by each object.
- Object class identification is recognised as a difficult process requiring a deep understanding of the application domain
- Object classes reflecting domain entities are reusable across systems
- Various object models may be produced, e.g. Inheritance models, Aggregation models, Interaction models

Inheritance models

- Organise the domain object classes into a hierarchy.
- Classes at the top of the hierarchy reflect the common features of all classes.
- Object classes inherit their attributes and services from one or more super-classes - these may then be specialised as necessary.
- Class hierarchy design can be a difficult process if duplication in different branches is to be avoided.
- Class diagrams - describe classes and their relationships

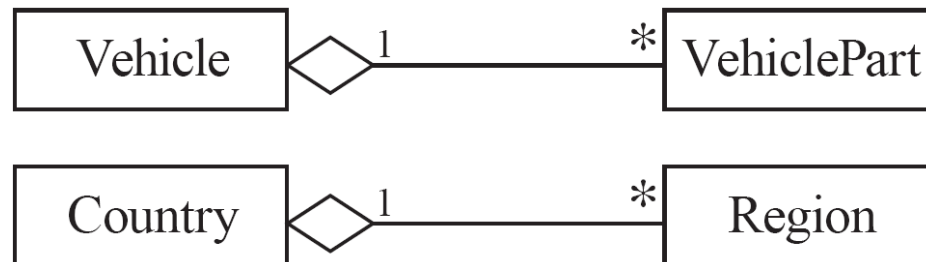
Library class hierarchy

Library class hierarchy

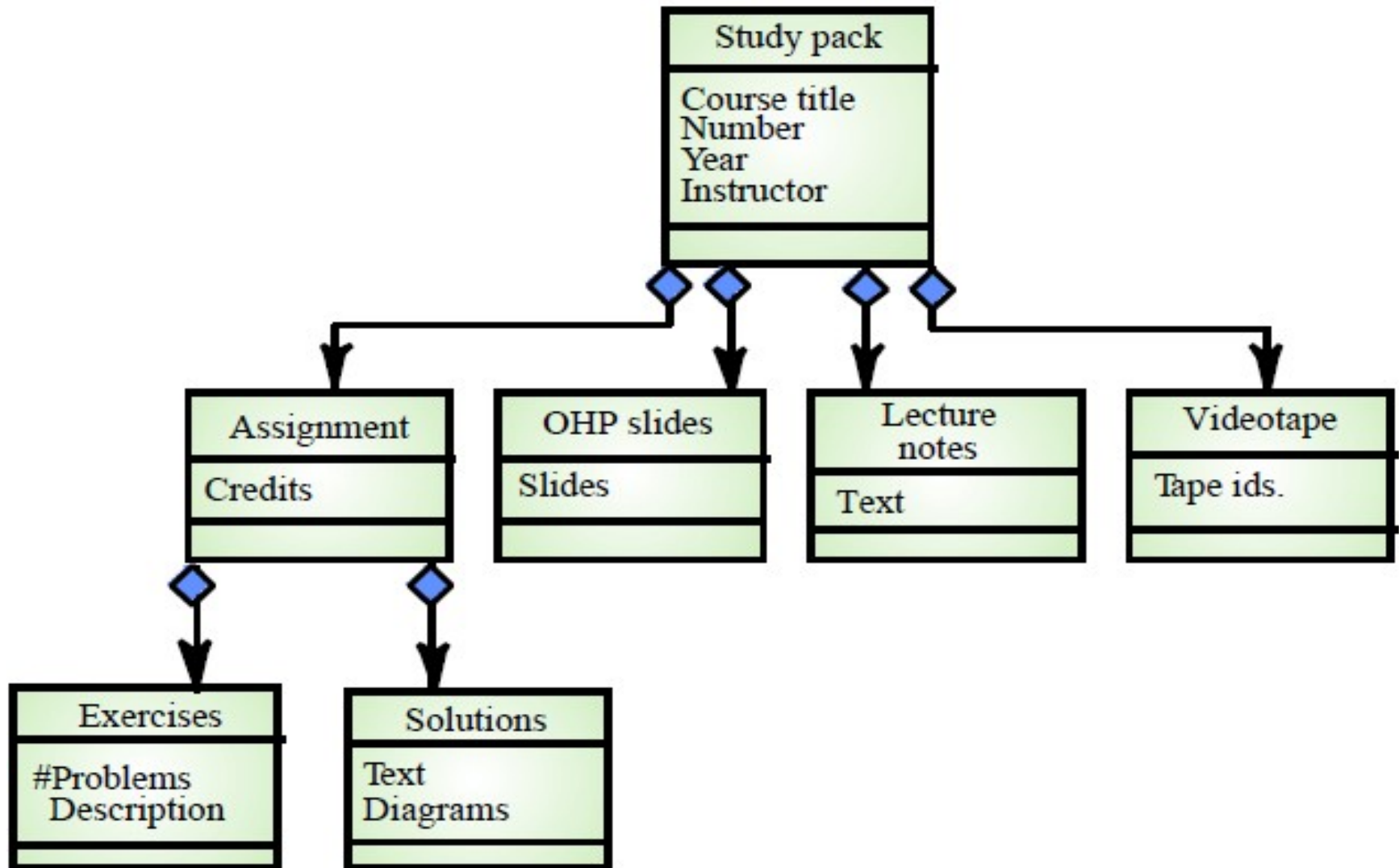


Object aggregation

- Is a special kind of association that represents ‘part-whole’ relationships.
- An aggregation model shows how classes that are collections are composed of other classes.
- Aggregation models are similar to the part-of relationship in semantic data models.
- The UML notation for aggregation is to represent the composition by including a diamond shape on the source of the link.



Object aggregation representing a course



Object behaviour modelling (Interaction models)

- A behavioural model shows the interactions between objects to produce some particular system behaviour that is specified as a use-case.
- Use-case
 - Describes what a system does from the standpoint of an external observer.
 - Emphasis on *what* a system does rather than *how*.
 - Cover the *full sequence of steps* from the beginning of a task until the end.
 - The objective of *use case analysis* is to model the system from the point of view of how users interact with this system when trying to achieve their objectives.
 - It is one of the key activities in requirements analysis

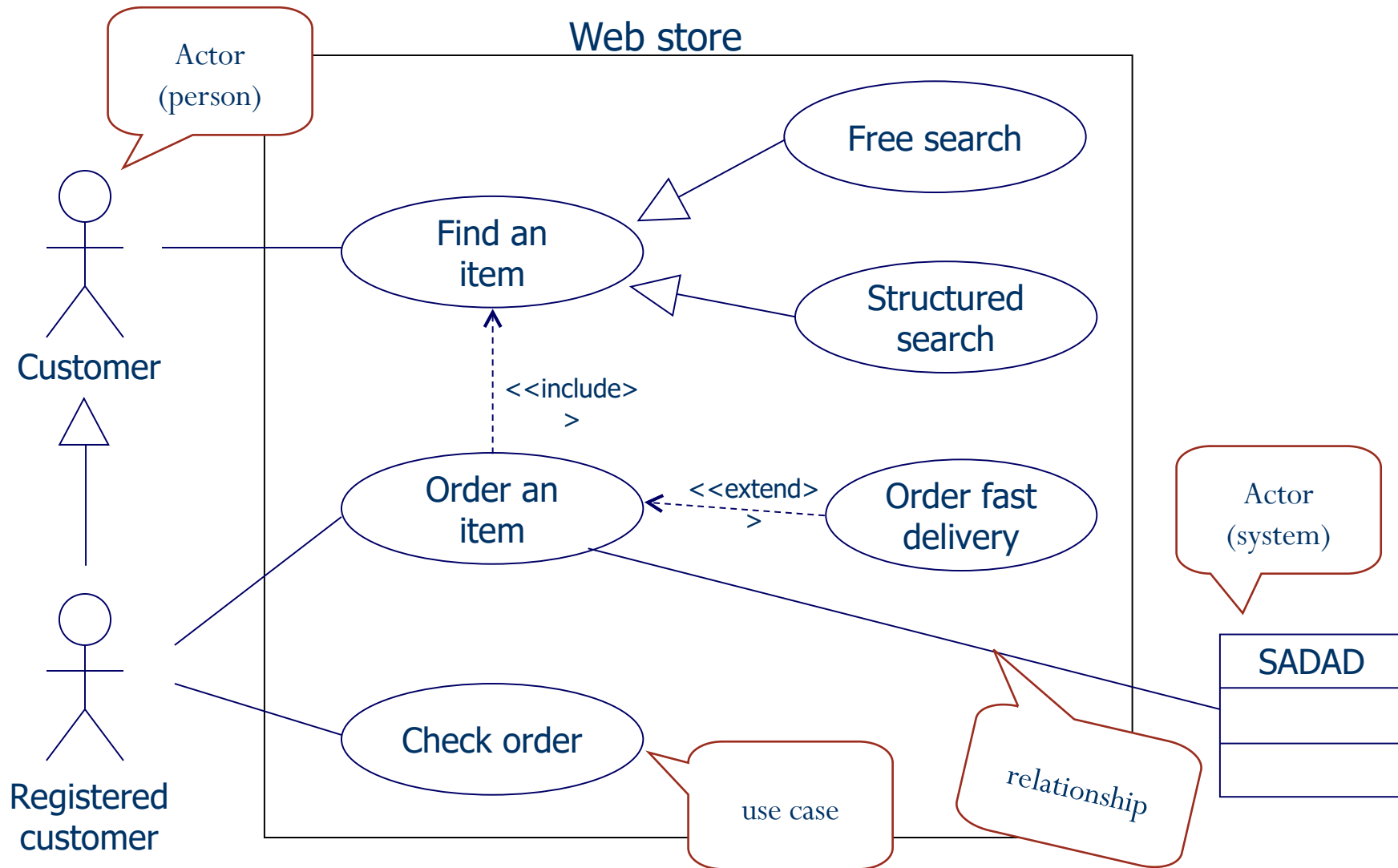
Use Case Diagrams

- Use case diagrams contain:
 - A set of *ACTORS* : roles users can play in interacting with the system.
 - An *actor* is a user of the system playing a particular role.
 - A set of *USE CASES*: each describes a possible kind of interaction between an actor and the system.
 - Uses cases are actions that a user takes on a system
 - A number of *RELATIONSHIPS* between Use Cases and Actors.
 - A dashed line between use cases is used to indicate these relationships.
 - There are three types of use case relationships: Inclusion, Extension and Generalizations

Use Case Diagrams-Relationships

- Inclusion <<includes>>
 - Inclusion enables to reuse one use case's steps inside another use case.
 - Allow one to express *commonality* between several different use cases.
- Extension <<extends>>
 - Allows creating a new use case by adding steps to existing use cases
 - The base and extended use cases are complete processes on their own.
- Generalization
 - Allows child actors to inherit behavior from parent actors
 - Much like super classes in a class diagram.

Example of use case diagram

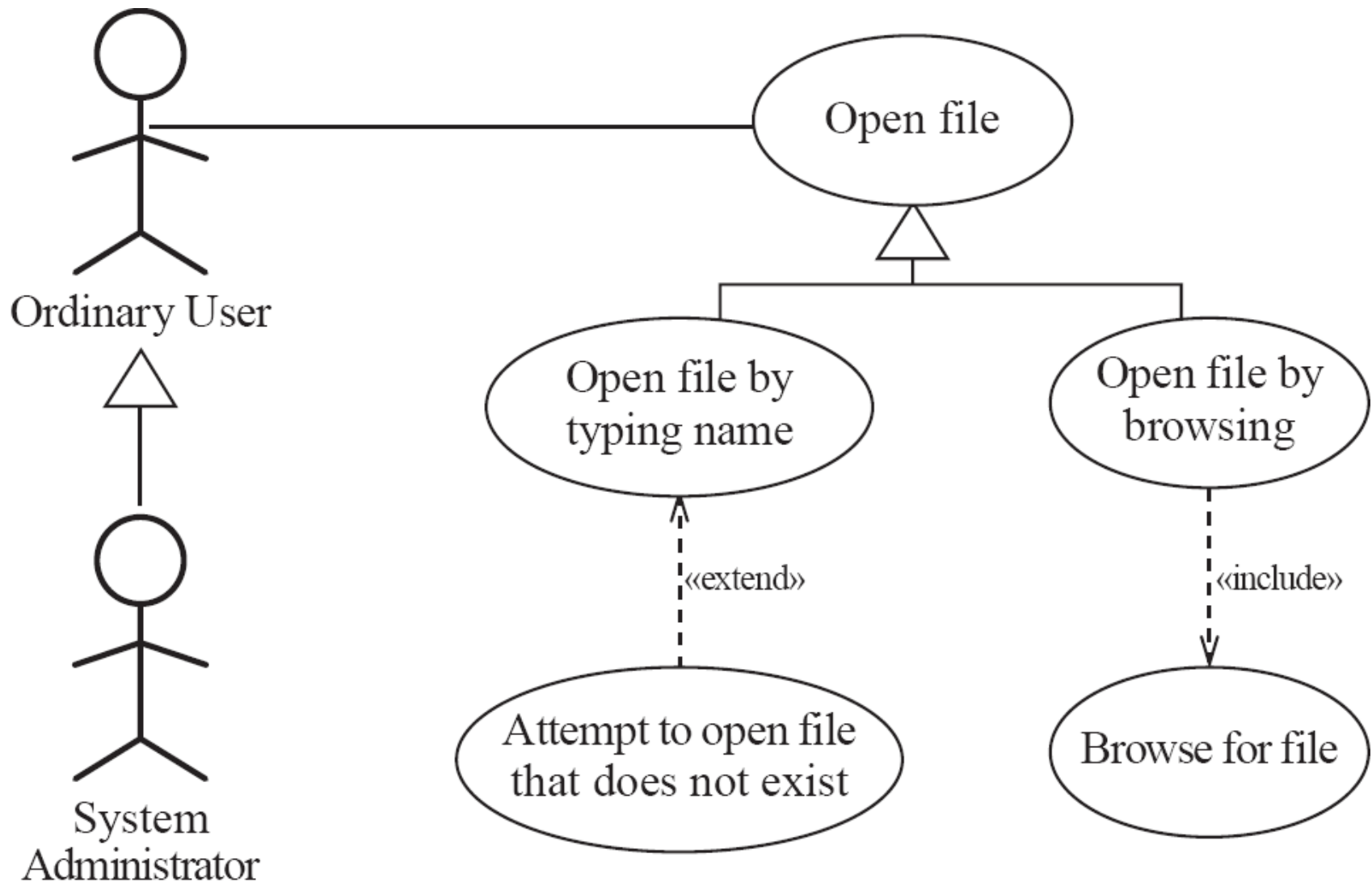


How to describe a single use case

- **Name:** Give a short, descriptive name to the use case.
- **Actors:** List the actors who can perform this use case.
- **Goals:** Explain what the actor or actors are trying to achieve.
- **Preconditions:** State of the system before the use case.
- **Summary:** Give a short informal description.
- **Related use cases.**
- **Steps:** Describe each step using a 2-column format.
- **Post conditions:** State of the system in following completion.

Name and steps are the most important

Use Case example



Example: description of a use case

Use case: Open file

Related use cases:

Generalization of:

- Open file by typing name
- Open file by browsing

Steps:

Actor actions

1. Choose 'Open...' command
3. Specify filename
4. Confirm selection

System responses

2. File open dialog appears
5. Dialog disappears

Example...

Use case: Open file by typing name

Related use cases:

Specialization of: Open file

Steps:

Actor actions

1. Choose 'Open...' command
- 3a. Select text field
- 3b. Type file name
4. Click 'Open'

System responses

2. File open dialog appears
5. Dialog disappears

Example...

Use case: Open file by browsing

Related use cases:

Specialization of: Open file

Includes: Browse for file

Steps:

Actor actions

1. Choose 'Open...' command
3. Browse for file
4. Confirm selection

System responses

2. File open dialog appears
5. Dialog disappears

Example...

Use case: Attempt to open file that does not exist

Related use cases:

Extension of: Open file by typing name

Actor actions

1. Choose 'Open...' command
- 3a. Select text field
- 3b. Type file name
4. Click 'Open'
6. Correct the file name
7. Click 'Open'

System responses

2. File open dialog appears
5. System indicates that file does not exist
- 8 Dialog disappears

Example...

Use case: Browse for file (inclusion)

Steps:

Actor actions

1. If the desired file is not displayed, select a directory
3. Repeat step 1 until the desired file is displayed
4. Select a file

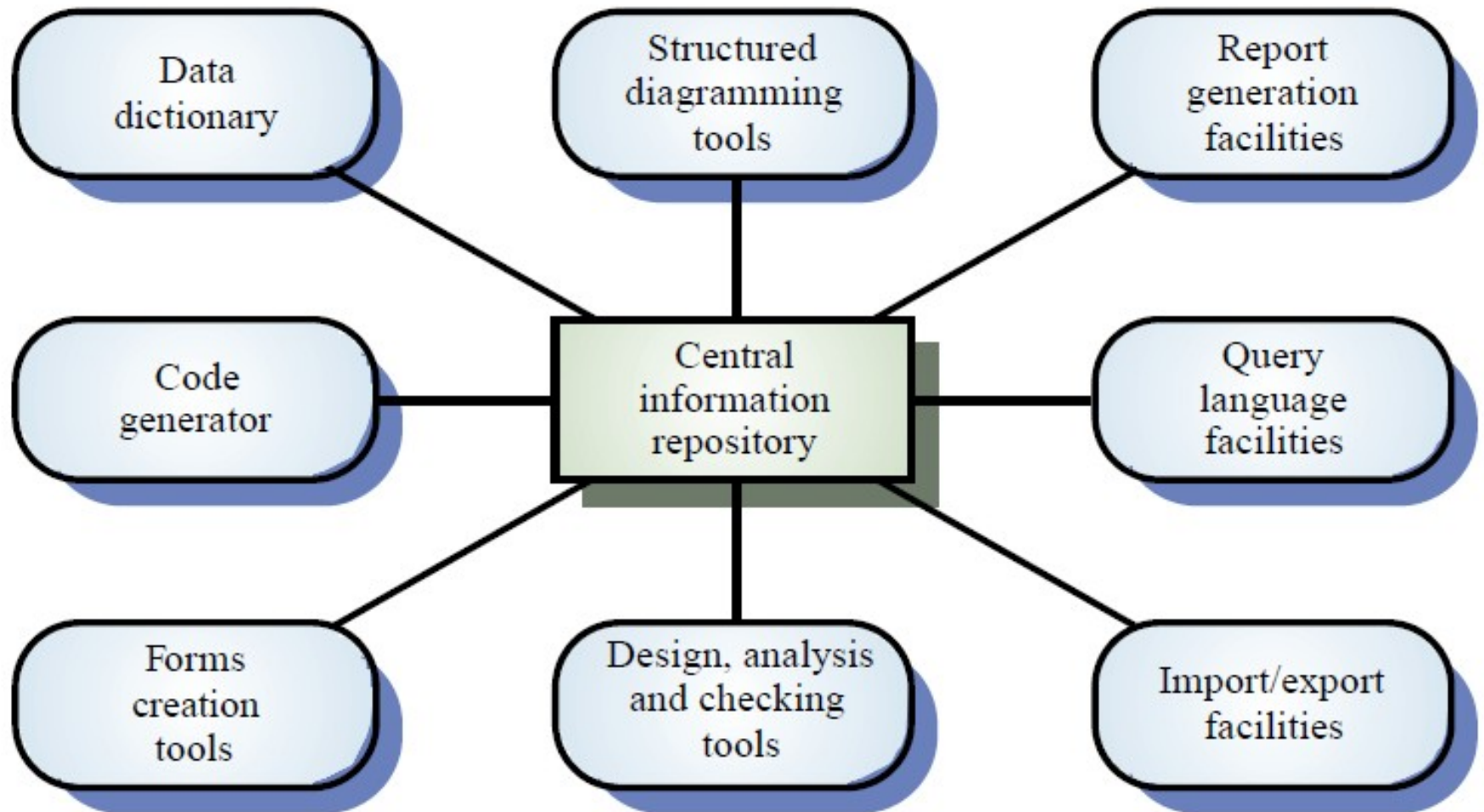
System responses

2. Contents of directory is displayed

CASE workbenches

- A coherent set of tools that is designed to support related software process activities such as analysis, design or testing.
- Analysis and design workbenches support system modelling during both requirements engineering and system design.
- These workbenches may support a specific design method or may provide support for a creating several different types of system model.

An analysis and design workbook



System models weaknesses

- They do not model non-functional system requirements
- They do not usually include information about whether a method is appropriate for a given problem
- They may produce too much documentation
- The system models are sometimes too detailed and difficult for users to understand